Mathematische Optimierung

Meine Ideen in der Konzipierung

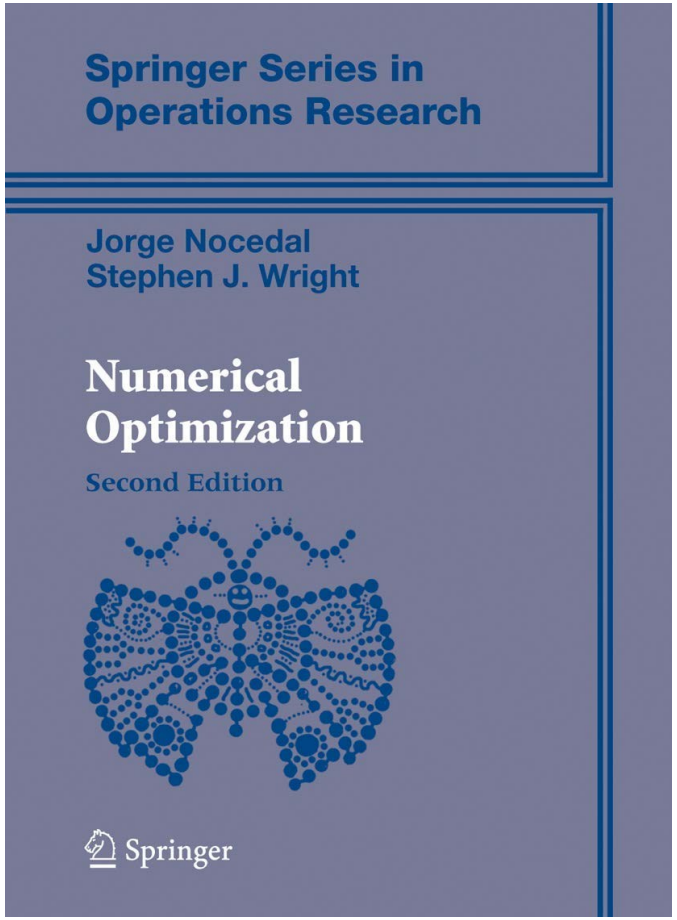
- Wir machen inhaltlich was spannendes
- Wir machen keine Klausur
  - Formal: 10% Abgabe, 90% mündliche Prüfung.
- Sie programmieren selber im Praktikum die Vorlesungsinhalte nach
  - Ich gebe Code mit Grundfunktionalität vor (bald in ILIAS).
  - Wenn der Code läuft, dann bestehen Sie.
  - „Besserer“ Code = bessere Note (Details später)
- Abgabe bis Ende Februar.
- Lernziele:
  - Typische Optimierungsverfahren und deren Einsatzgebiete und Anwendungen
  - Softwareentwicklungen im mathematischen Kontext

Termine

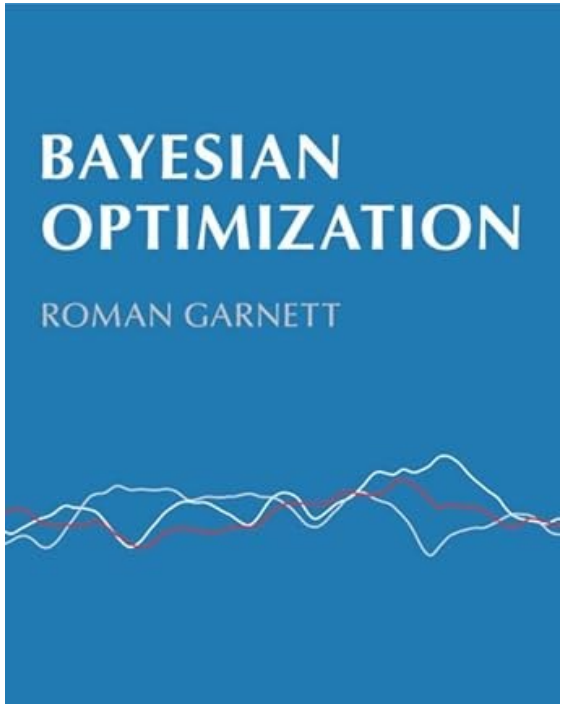
- Zu Beginn mehr Vorlesungen.
- Später mehr Praktika
- Sobald Vorlesungen durch sind, findet nur noch das Praktikum statt.
- Wann können Sie?
  - Mo, 8:00 (ggf. WPFs) -> 19.10. möglich?
  - Mo, 9:45 (sollten alle können) -> heute, 12.10., 19.10.
  - Di, 8:00 (E hat RT) -> 6.10. und/oder 13.10. und/oder 20.10. möglich?
  - Mi, 9:45 (eher mehr WPFs) -> 14.10. und 21.10. möglich?

Grundlagen

Quellen



- Wikipedia
- ChatGPT
- https://ncatlab.org/nlab/show/HomePage



Wo ist Optimierung?

- Wir wollen Anlagen, Prozesse, Produkte, Portfolios, Kalibrierungen, Software, Datenstrukturen und vieles Andere möglichst optimal wählen.
- Vielen dieser Fälle hängen von der Wahl endlich vieler (d) Zahlen ab, beispielsweise der Durchmesser einer Leitung in einer Anlage, die Taktzahl eines Prozesses oder der Temperatur von chemischen Prozessen.
- Bezeichne die Menge der möglichen Wahlen dieser Zahlen mit $X \subseteq \mathbb{R}^d$.
  - X ist eine Teilmenge von $\mathbb{R}^d$, weil Beispielsweise Taktzahlen positiv sind.
- In vielen dieser Fälle gilt es irgendwelche sich daraus ergebenden (reelle) Zahl (Ausbeute, Fehlerrate, etc.) optimal zu gestalten.
- Es gibt eine **Zielfunktion** $f: X \to \mathbb{R}$ und wir suchen $x \in X$ mit $f(x)$ optimal.

Konventionen

- Wir suchen ein $x \in X$, so dass $f(x)$ „optimal“ ist.
- „Optimal“ heißt hier (per Konvention) „Minimal“.
- Sollten wir f maximieren wollen, können wir auch $-f$ minimieren.

Was wird optimiert?

- In der Anwendung sind oft technische Geräte zu optimieren. Auf diese brauchen wir Zugriff über eine der folgenden beispielhaft genannten Möglichkeiten:
  - Mathematisch/physikalisches Modell (in einfachen Fällen, oft werden Parameter durch Messungen bestimmt)
  - Physikalisches Simulationsmodell (Simulink, numerische Simulationen, …)
  - Datengetriebenes Modell (Neuronale Netze, etc., oft basierend auf Messungen)
  - Optimierung am realen Prozess (Direktzugriff auf Hardware, Prototypen werden gebaut, …)

Mathematische Abstraktion der Optimierung

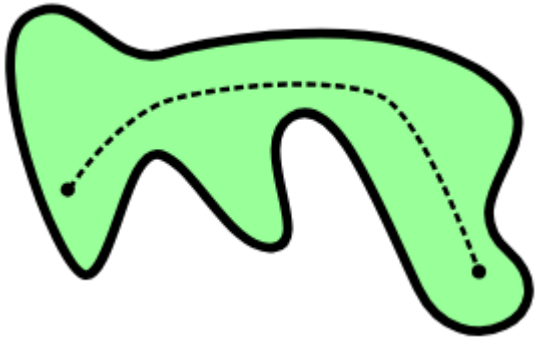
- In jedem Fall haben wir eine konkrete (aber jeweils andere) Datenstruktur für unser $f: X \to \mathbb{R}$ und können f auswerten.
  - Beachte: Auswertungen von f können teuer sein!
- Wir nennen hierbei X die **zulässige Menge** und jedes $x \in X$ einen **zulässigen Punkt**.
- Wir bezeichnen mit f die **Zielfunktion.**
- Ein $x \in X$ mit $f(x) \leq f(x')$ für alle $\mathrm{x}' \in X$ nennen wir **(globalen) Optimalpunkt** oder **(globale) Optimalstelle** oder **(globales) Optimum** und in diesem Fall nennen wir $f(x)$ den **(globalen) Optimalwert**.

Lokale Fragen

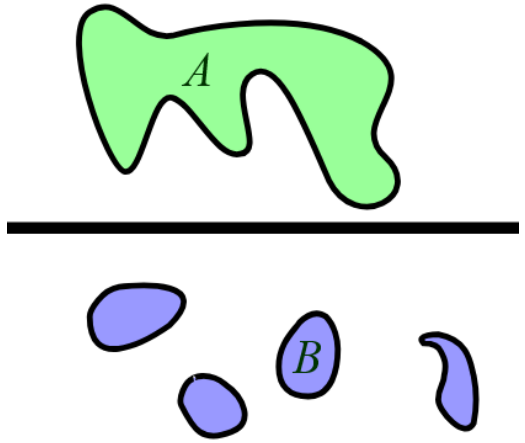
- In vielen Fällen ist es zu schwer oder ganz unmöglich, ein globales Optimum zu finden.
  - Wir greifen auf die übliche Vereinfachung aus Mathe2 zurück.
- Ein $x \in X$ nennen wir **lokalen Optimalpunkt** oder **lokale Optimalstelle** oder **lokales Optimum** falls es ein $\varepsilon > 0$ gibt mit $f(x) \leq f(x')$ für alle $\mathrm{x}' \in X$ mit $\left|\left|x' - x\right|\right| < \varepsilon$. In diesem Fall nennen wir $f(x)$ den **lokalen Optimalwert**.
- Wenn f differenzierbar ist (was wir für Funktionen $f: \mathbb{R}^d \to \mathbb{R}$ noch definieren müssen), dann wird sich das Finden von lokalen Optima als machbar herausstellen.
- Wenn man hinreichend viele lokale Optima findet, dann ist da hoffentlich das globale Optimum dabei. Dies nennt man **Multistart Optimierung**.

Zusammenhängende Mengen

- Wir nennen eine Menge X **zusammenhängend** (genauer: **pfadzusammenhängend** oder **wegzusammenhängend**), falls es für jede $x, x' \in X$ einen eine stetige Funktion (genannt: **Pfad** oder **Weg**) $p: [0,1] \to X$ mit $p(0) = x$ und $p(1) = x'$ gibt.
- (Wir müssten noch formal Stetigkeit definieren, da sind aber keine Überraschungen)



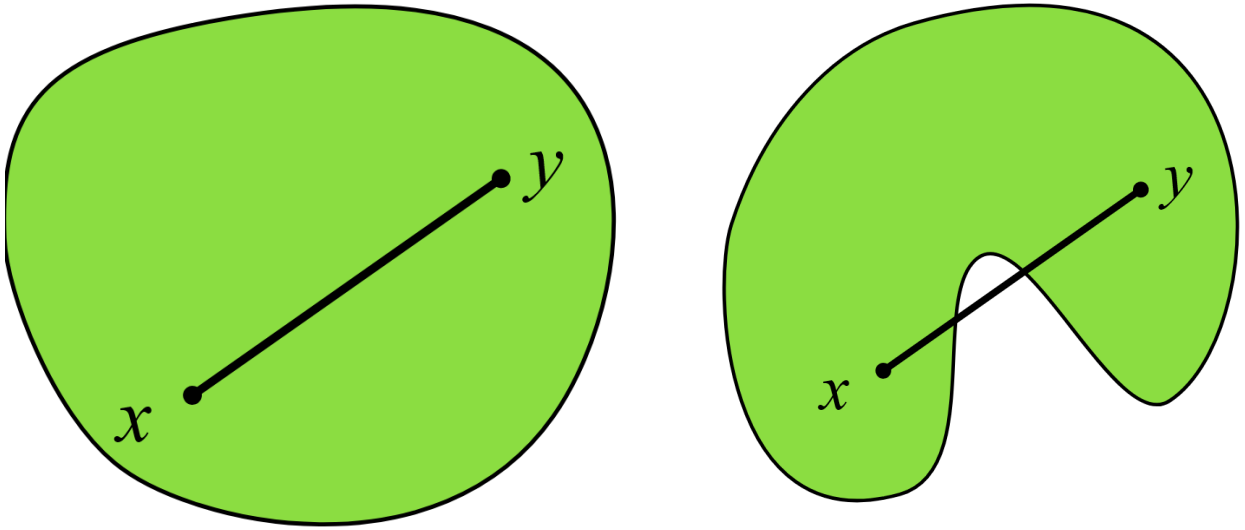
https://upload.wikimedia.org/wikipedia/commons/b/b8/Path-connected_space.svg, gemeinfrei

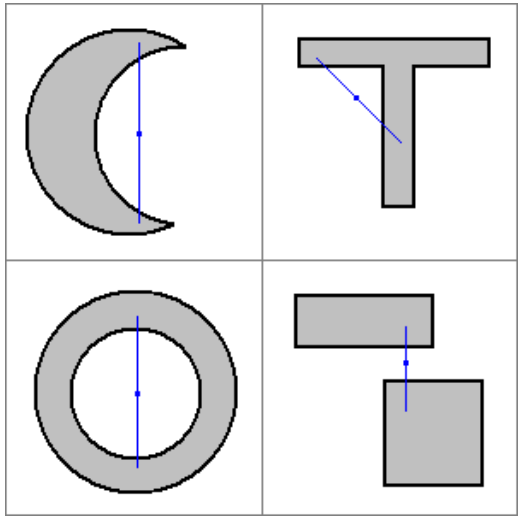


https://upload.wikimedia.org/wikipedia/commons/9/95/Connected_and_disconnected_spaces.svg , gemeinfrei

Konvexe Mengen

- Wir nennen eine Menge **konvex**, wenn für jede $x, x' \in X$ einen die Verbindungsstrecke $\{\lambda x + (1-\lambda)x' | \lambda \in [0,1]\}$ eine Teilmenge von X ist.





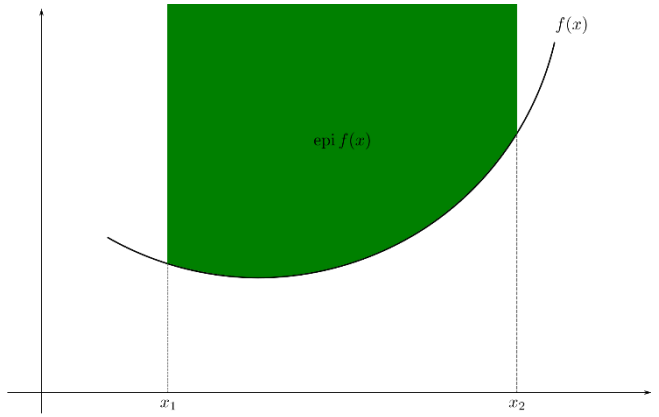
https://upload.wikimedia.org/wikipedia/commons/5/5f/Convex_set.svg, gemeinfrei

https://upload.wikimedia.org/wikipedia/commons/d/dc/KonkaveFiguren.png, CC BY-SA 3.0

https://upload.wikimedia.org/wikipedia/commons/1/1f/Non_Convex_set.svg, gemeinfrei

Fakten über konvexe Mengen

- Konvexe Mengen sind zusammenhängend
  - Sehr leicht zu beweisen: der Pfad ist die Strecke
- Bei einer konvexen Funktion ist die Menge oberhalb des Funktionsgraphen Konvex



https://de.wikipedia.org/wiki/Datei:Epigraph_convex.svg, CC BY-SA 3.0

Datenstrukturen für Mengen

- Wir wollen alles (ok, theoretisch alles, praktisch vieles) von dem, was wir hier machen in Software übersetzen.
- Es stellt sich also immer wieder die Frage, was geeignete Datenstrukturen für gewisse mathematische Objekte sind.
- Das wird später noch ein wichtiges und wiederkehrendes Thema sein, auch in Bezug auf Codequalität.
- Wir geben hier (wenige) typische Datenstrukturen für Mengen an.

Mehrdimensionale Intervalle

- Die Mengen der Form $X = [l_1, u_1] \times \cdots \times [l_d, u_d] \subset \mathbb{R}^d$ für $l_i \leq u_i$, beide in $\mathbb{R}$ für alle d, nennt man **mehrdimensionale Intervalle** (oder auch **Hyperquader** oder **Produkte von Intervallen** oder **Hyperrechtecke**). Englisch oft **Bounds**.
- Mehrdimensionale Intervalle sind zusammenhängend und konvex.
- Die sind ganz typische zulässige Mengen: d Werte können jeweils in einem individuell vorgegebenen Bereich eingestellt werden.

Ungleichheitsconstraints (Inequality Constraints)

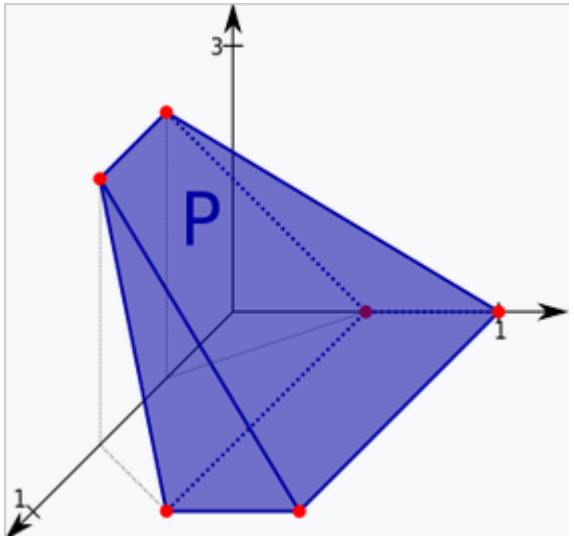
- Mengen der Form $X = h^{-1}((-\infty, 0]) = \{x \in \mathbb{R}^d \big| h(x) \leq 0\} \subseteq \mathbb{R}^d$ für $h: \mathbb{R}^d \to \mathbb{R}$ nennt man beschrieben durch eine **Ungleichheitsbedingung** (**Inequality Constraint**)
- Beispiel $h: \mathbb{R}^2 \to \mathbb{R}: (x_1, x_2) \mapsto x_1^2 + x_2^2 - 1$
- Hierdurch bleibt typischerweise die Dimension (wäre zu definieren, ist aber hoffentlich anschaulich klar) gleich d.
- Bei solche Mengen ist es sowohl möglich, dass sie zusammenhängend oder konvex sind, oder auch nicht.

Gleichheitsbedingung (Equality Constraints)

- Mengen der Form $X = g^{-1}(\{0\}) = \{x \in \mathbb{R}^d \big| g(x) = 0\} \subseteq \mathbb{R}^d$ für $g: \mathbb{R}^d \to \mathbb{R}$ nennt man beschrieben durch eine **Gleichheitsbedingung** (**Equality Constraint**)
- Beispiel $g: \mathbb{R}^2 \to \mathbb{R}: (x_1, x_2) \mapsto x_1^2 + x_2^2 - 1$
- Hierdurch wird typischerweise die Dimension (wäre zu definieren, ist aber hoffentlich anschaulich klar) kleiner als d.
- Bei solche Mengen ist es sowohl möglich, dass sie zusammenhängend oder konvex sind, oder auch nicht. Normalerweise sind sie es nicht.

Lineare Constraints

- $Ax \leq a$ für $A \in \mathbb{R}^{c \times d}$ und $a \in \mathbb{R}^{c}$ (**Konvexe Polytope**)
- $Bx = b$ für $B \in \mathbb{R}^{c \times d}$ und $b \in \mathbb{R}^{c}$ (**Affine Teilräume**, bspw. Ebenen)



https://en.wikipedia.org/wiki/Convex_polytope#/media/File:3dpoly.svg, public domain

Schnitte

- Schnitte von Mengen sind wieder Mengen. Wir können also die obigen Constraints kombinieren, bspw:

$$X = \left\{ x \in \mathbb{R}^d \middle| l_i \leq x_i \leq u_i, h_j(x) \leq 0, g_k(x) = 0, 1 \leq i \leq d, 1 \leq j \leq j_g, 1 \leq k \leq k_h \right\}$$

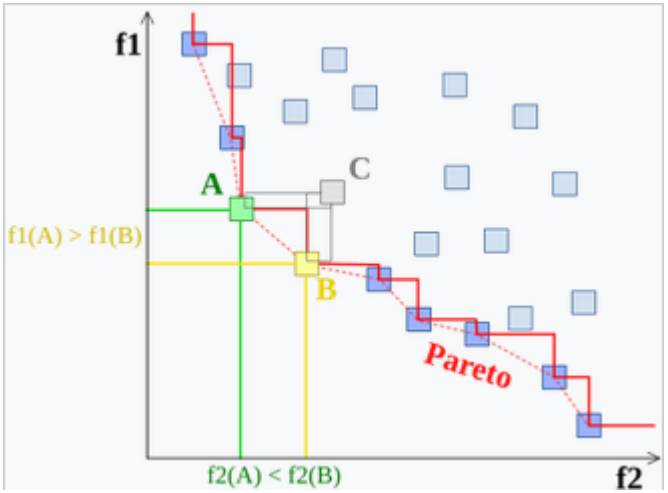
- Beachte: alle Datentypen für Mengen lassen sich durch Equality und Inequality Constraints beschreiben. Für die spezielleren Datentypen gibt es aber oft bessere Algorithmen.
- Wir werden in unserer Lehrveranstaltung den Fokus auf Bounds und Inequality Constraints legen.

Weitere Optimierungsziele: Rauschen

- Die Zielfunktion $f: \mathbb{R}^d \rightarrow \mathbb{R}$ ist keine Funktion, sondern liefert stochastische/verrauschte Werte.
- Gerade bei der Optimierung von technischem Equipment direkt in Hardware ist dies der Standard.
- Von unseren Methoden ist nur die Bayesian Optimization direkt in der Lage, mit Rauschen umzugehen.

Weitere Optimierungsziele: Pareto

- Die Zielfunktion $f: \mathbb{R}^d \rightarrow \mathbb{R}^e$ bildet auf mehrere zu optimierende Ziele ab.
- In dem Fall hat man kein eindeutiges Optimum mehr, sondern nur noch eine Pareto-Front.
- Die Pareto-Front besteht aus allen $f(x)$, so dass es kein $x' \in X$ gibt mit $f(x') \leq f(x)$, wobei die Ungleichheit komponentenweise zu sehen ist.
- Viele Optimierungsalgorithmen lassen sich hierfür abwandeln
- Pareto-Optimierung ist für große e schwer



https://de.wikipedia.org/wiki/Pareto-Optimum#/media/Datei:Front_pareto.svg, CC BY-SA 3.0

Kommentare

- Bei der Erstellung von Optimierungsalgorithmen gibt es verschiedene KPIs, welche zu beachten sind. Wie so oft, sind diese teilweise gegenläufig:
  - Numerische Genauigkeit, mit der ein lokales Optimum gefunden wird
  - Zuverlässigkeit, mit der ein globales Optimum gefunden wird
  - Rechenzeit des Optimierungsalgorithmus
  - Anzahl der Funktionsaufrufe (und Aufrufe der Ableitungen der Funktion)
  - Vermeidung von numerischen Fehlern in der Optimierung
  - Einfachheit der Bedienung
  - Lesbarkeit des Codes
  - Wartbarkeit des Codes
  - Testbarkeit des Codes
- Auf diese Punkte werden wir wiederholt eingehen und sie werden wichtige Teile des Praktikums sein.

Optimierung Mehrfach

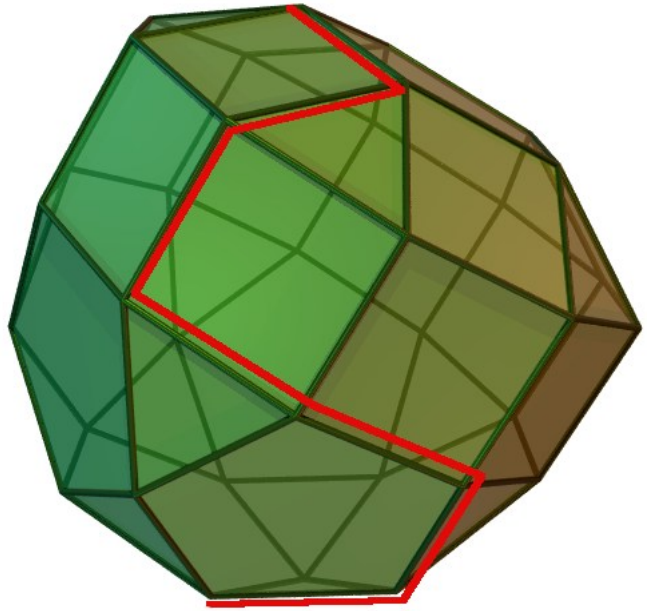
- In einem typischen Workflow kann die Optimierung mehrfach vorkommen:
  - Design of Experiment ist eine Optimierung der Messstellen
    - (Moderner auch aktives Lernen)
  - Optimierung bei der Modellerstellung, bspw. das Training von neuronalen Netzen
    - Ggf. auch Optimierung der Hyperparameter
  - Optimierung eines Prozesses/Produktes mittels des Modells
  - Ggf. Nachoptimierung am Prozess
- Wir werden nur beschränkt auf diese Unterschiede eingehen. Es werden oft ähnliche Optimierungsalgorithmen verwendet.

~~Dumme~~ Suboptimale Verfahren

- Full Grid
  - Schneide Gitter (Lattice, nicht Grid) in $\mathbb{R}^d$ mit X und wähle das Optimum dieser Punkte
  - Braucht extrem (exponentiell in d) viele Funktionsauswertungen
  - Ist trotzdem nicht sehr genau
- Iterativ 1D
  - Algorithmus:
    - 1) Wähle Startwert.
    - 2) Optimiere 1. Koordinate
    - …
    - d+1) Optimiere d-te Koordinate
    - d+2) Solange noch Rechenbudget da ist, gehe zu 2)
  - Nicht sehr genau
  - Braucht lange
  - Schlecht, wenn der Gradient den Winkel $\frac{\pi}{4}$ zu den Koordinatenachsen hat

Das Simplexverfahren in Polytopen

- Es gibt zwei Optimierungsverfahren namens Simplexverfahren.
- Eines davon: man optimiert eine lineare Zielfunktion unter affinen Nebenbedingungen
- Dabei läuft man den Rand eine Polytops ab



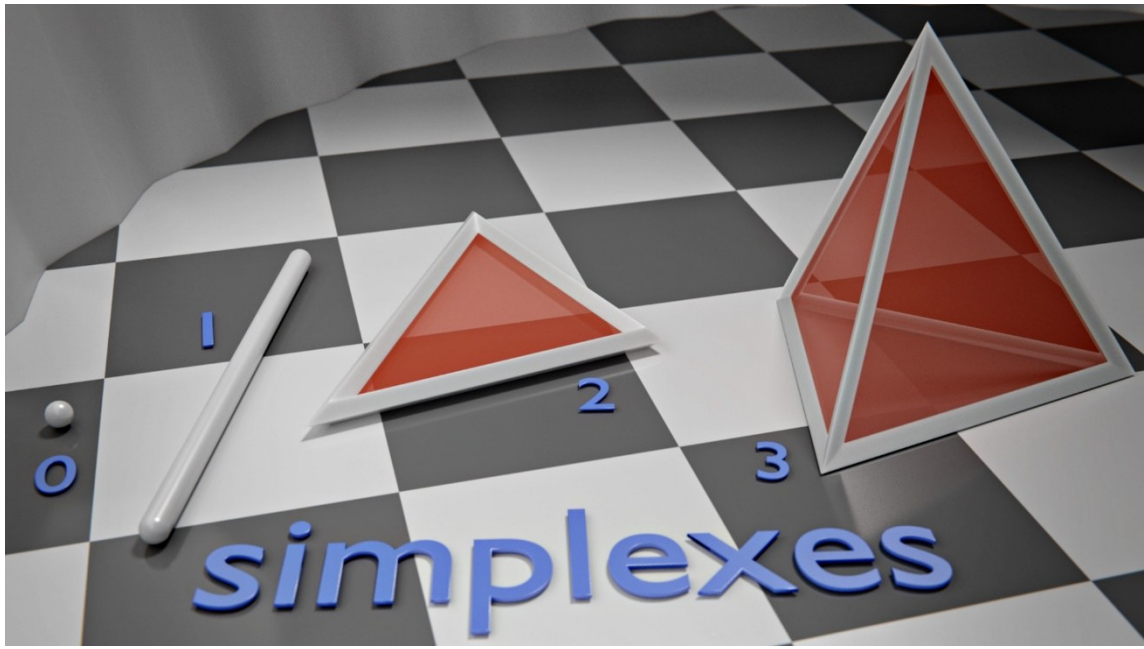
https://de.wikipedia.org/wiki/Simplex-Verfahren#/media/Datei:Simplex-method-3-dimensions.png, CC BY-SA 3.0

Das Simplexverfahren
Als Optimierungsverfahren ohne Ableitung

- Wir betrachten das andere Simplexverfahren genauer.
- Es heißt auch **Downhill-Simplex-Verfahren** oder **Nelder-Mead-Verfahren**
- Das Verfahren sucht lokale Optima.
- Besonderheit: es verwendet keine Ableitungen.
- Dies macht es relativ universell einsetzbar, aber auch langsam.
- Wir betrachten das Simplex-Verfahren zur Motivation.

Simplexverfahren

- Wir gehen davon aus, dass in $X = \mathbb{R}^d$ optimiert wird.
- Wähle $d + 1$ Punkte $x_0, \dots, x_d \in X = \mathbb{R}^d$ als Startpunkte.
  - Diese Punkte müssen affin unabhängig sein, d.h. die $x_i - x_0$ müssen linear unabhängig sein.
  - Diese Punkte bilden daher einen Simplex.
  - Dies ist eine Schleifeninvariante
- Sortiere diese Punkte immer mit $f(x_0) \leq f(x_1) \leq \cdots \leq f(x_d)$.
- Wir werden x_d iterativ durch einen besseren Punkt ersetzen.



https://en.wikipedia.org/wiki/Simplex#/media/File:Simplexes.jpg, CC BY-SA 4.0

Simplexverfahren

- Bilde den Mittelpunkt $\overline{x} = \frac{1}{d}\sum_{i=0}^{d-1} x_i$ aller Punkte außer dem schlechtesten x_d
- Reflektiere x_d an $\overline{x}$: $x' = \overline{x} + \alpha(\overline{x} - x_d)$
- Wenn $f(x') < f(x_0)$:
  - Expandiere: $x'' = x' + \gamma(x' - \overline{x})$
  - Beginne die nächste Iteration mit $x_d = argmin(f(x)|x \in \{x', x''\})$
- Wenn $f(x') < f(x_{d-1})$
  - Beginne die nächste Iteration mit $x_d = x'$
- Sei $\hat{x} = argmin(f(x)|x \in \{x', x_d\})$
- Kontrahiere: $\tilde{x} = \hat{x} + \beta(\overline{x} - \hat{x})$
- Wenn $f(\tilde{x}) < f(x_d)$
  - Beginne die nächste Iteration mit $x_d = \tilde{x}$
- Komprimiere
  - Beginne die nächste Iteration mit $x_i = x_i + \delta(x_0 - x_i)$ für $1 \leq i \leq d$

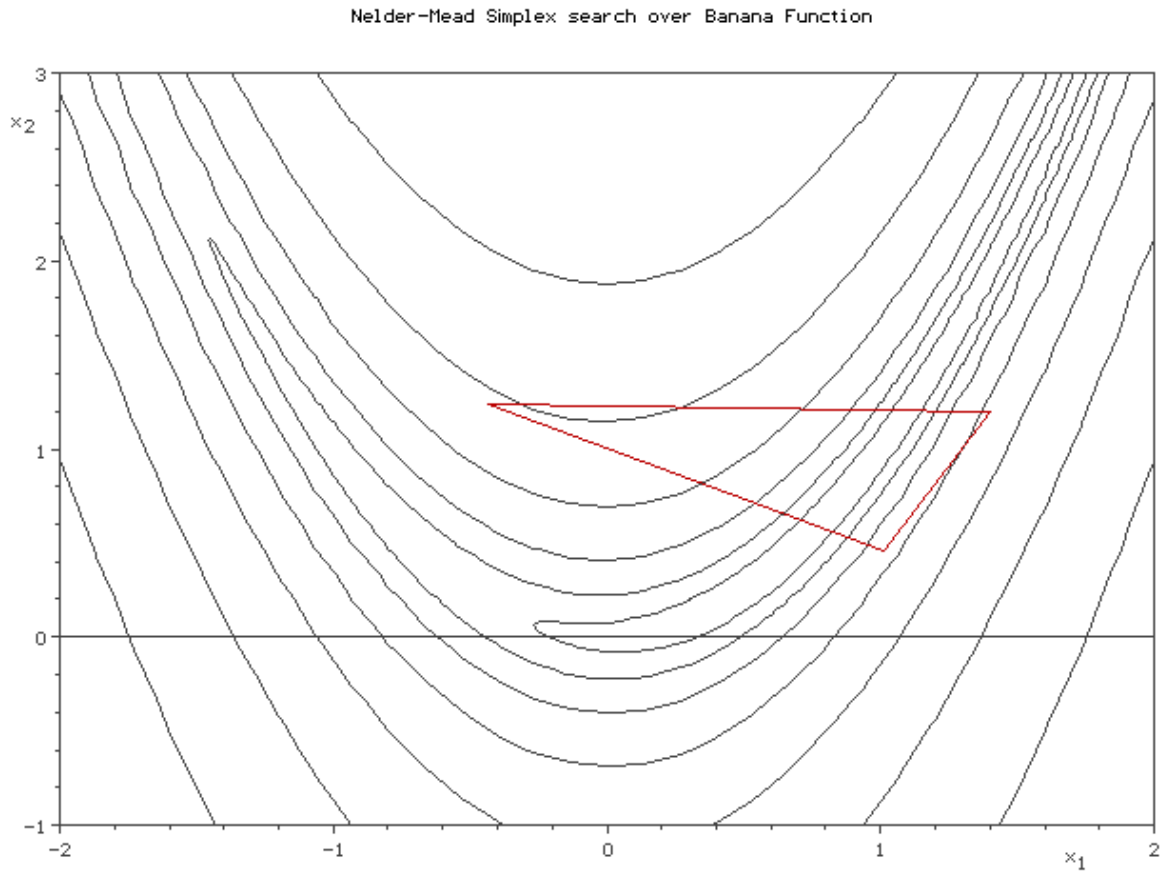
4 Parameter

- Reflexion $\alpha > 0$, oft $\alpha = 1$
- Expansion $\gamma > 0$, oft $\gamma = 2$
- Kontraktion $\beta \in (0,1)$, oft $\beta = \frac{1}{2}$
- Komprimierung $\delta \in (0,1)$, oft $\delta = \frac{1}{2}$

Terminierung

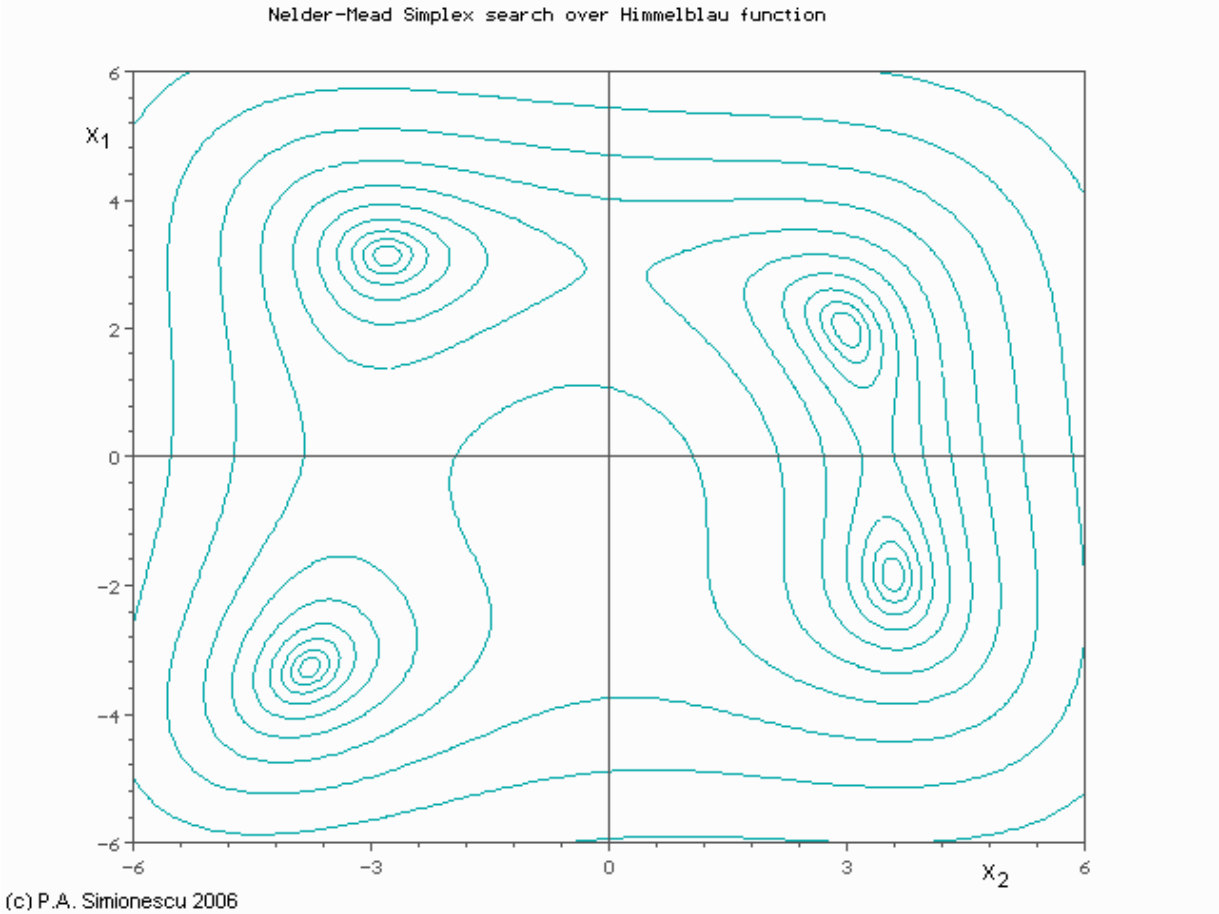
- Wenn maximale Anzahl an Iterationen erreicht ist.
- Oder wenn Simplex „klein genug“.
- Funktioniert oft gut, aber wenige theoretische Garantien.
- Rückgabewert ist oft x_0.
- Bei konvexen Funktionen (was auch immer das in mehreren Dimension ist) gibt es die Garantie, dass die Folge der x_0 zu immer kleineren Funktionswerten führt.

Beispiellauf mit Rosenbrock-„Bananenfunktion“



https://de.wikipedia.org/wiki/Downhill-Simplex-Verfahren#/media/Datei:Nelder_Mead1.gif, CC BY-SA 3.0

Beispiellauf mit Himmelblau-Funktion



https://de.wikipedia.org/wiki/Downhill-Simplex-Verfahren#/media/Datei:Nelder_Mead2.gif, CC BY-SA 3.0

Kommentare

- Simplizes werden anfänglich eher groß gewählt, um ein großes Gebiet abzudecken
- Ggf. wird manchmal x_d durch einen zufällig Punkt ersetzt, um aus lokalen Extrema herauszukommen
- Die Verwendung mit Randwerten projiziert den neuen Simplexpunkt oft in den erlaubten Bereich zurück
  - Falls notwendig mit einer Variante von Line Search, siehe später.
- Vorteil: man kommt ohne Ableitungen aus
  - Aber heute haben wir typischerweise Ableitungen für vieles (wenn ein aktueller Software-Stack verwendet wird).

Lernziele und Bewertung

- Lernziele:
  - Typische Optimierungsverfahren und deren Einsatzgebiete und Anwendungen
  - Softwareentwicklungen im mathematischen Kontext
- Bewertung:
  - Sie geben Programmieraufgaben in Python ab
  - Die Lösung wird primär nach 3 Kriterien beurteilt:
    - **Effizienz**: gemessen in Laufzeit und Auswertungen von Funktionswerten und Gradienten der Zielfunktion. Kommentiere jeweils O-Notation, Entscheidungen für bessere Effizienz, resultierende Laufzeiten und Auswertungen,…
    - **Wartbarkeit**: baue Implementierung geeignet in die vorhandene Codebasis ein, mache geeignete und umfangreiche Tests, halte die Anzahl der Codezeilen kurz,…
    - **Lesbarkeit**: Kommentiere wichtige Ideen, wähle geeignete Strukturen, separiere Ideen im Code sind,…
  - **Begründen** Sie dabei immer kurz, warum Sie gewisse Entscheidungen getroffen haben.
  - **Vergleichen** Sie Optimierungsergebnisse und Rechengeschwindigkeiten zwischen existierenden und neuen Implementierungen.
  - Machen Sie sich **selber Gedanken**, wie Ihre Lösung **hilfreich** sein kann, ich werde signifikant Punkte für minimalistische Lösungen abziehen.